ПРИМЕЊЕНО СОФТВЕРСКО ИНЖЕЊЕРСТВО
ЕЛЕМЕНТИ РАЗВОЈА СОФТВЕРА

Захтеви, backlog, Git и тимски развој

Од захтева и backlog ставке до проверене промене у репозиторијуму.



Факултет техничких наука · Универзитет у Новом Саду

Нови Сад

ПРЕГЛЕД

Садржај презентације

01

Agile и Scrum

Вредности, улоге, Sprint, Daily Scrum, артефакти, Ready и Done.

04–12

02

Захтеви

Корисничка прича, INVEST, критеријуми прихватања и Given–When–Then.

13–17

03

Backlog и табла

Садржај ставке, разлагање задатка, стања рада и Tapiz Boards.

18–22

04

Git

Појмови, команде, гране, Pull Request, GitHub Desktop и Visual Studio.

23–53

05

Предаја и КТ1

Контролна тачка, AI алати, провера пре предаје и припрема.

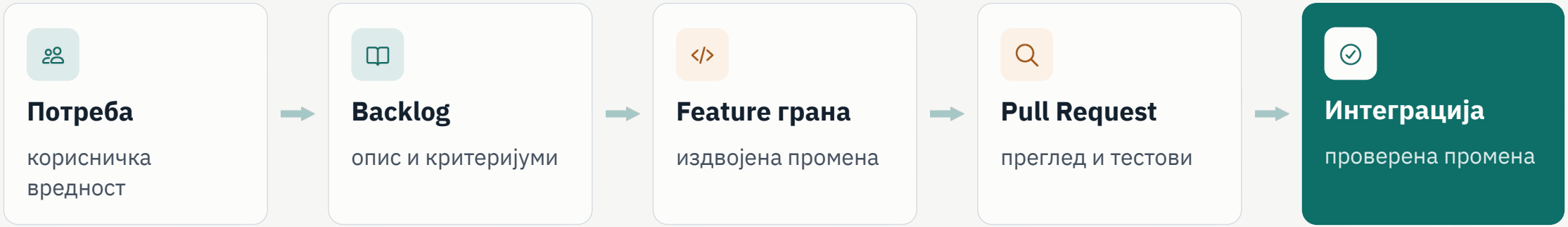
54–60

Увод · развојни ток од захтева до интеграције (03)

УВОД

Развојни ток од захтева до интеграције

Захтев, планирање рада и Git чине повезан процес.



НАПОМЕНА

Развојни ток повезује захтев, одлуке, имплементацију и доказе провере.

ПРВИ ДЕО

01

Agile и Scrum

Вредности агилног развоја и Scrum оквир у коме тим планира, ради и редовно проверава резултат.

- Agile вредности
- Scrum оквир
- Улоге
- Sprint и догађаји
- Daily Scrum
- Review и ретроспектива
- Артефакти
- Ready и Done

01 · AGILE И SCRUM

Вредности агилног развоја

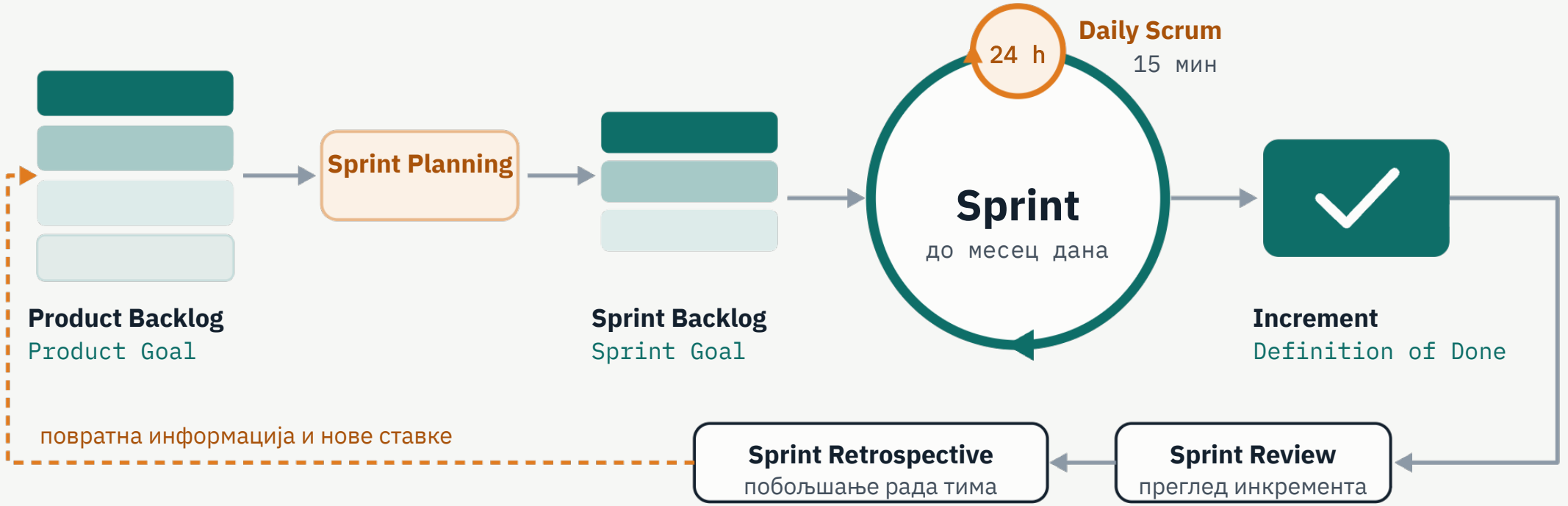
Појединци и интеракције	више од	процеса и алата
Софтвер који ради	више од	исцрпне документације
Сарадња са корисником	више од	преговарања о уговору
Реаговање на промене	више од	праћења плана

- Кратки циклуси и чест увид у резултат
- Захтеви се прецизирају током рада
- Софтвер који ради је мера напретка

Ставке десно имају вредност, али се ставке лево вреднују више. Извор: Manifesto for Agile Software Development, 2001.

Scrum оквир у једној слици

Тим ради у циклусима фиксног трајања, а сваки циклус завршава проверљивим инкрементом.



01 · AGILE И SCRUM

Улоге у Scrum тиму

Свака улога носи јасну одговорност, а тим заједно одговара за резултат.

Product Owner

`вредност производа`

Формулише Product Goal.

Уређује ставке и приоритете у Product Backlog-у.

Обезбеђује да је backlog јасан целом тиму.

Scrum Master

`ефикасност тима`

Помаже тиму да разуме и примењује Scrum.

Уклања препреке које успоравају рад.

Брине да се догађаји одрже и буду корисни.

Developers

`исправан Increment`

Планирају Sprint Backlog.

Свакодневно прилагођавају план ка Sprint Goal-у.

Поштују Definition of Done.

Scrum тим обично има 10 или мање чланова, без подтимова и хијерархије. Извор: The Scrum Guide, 2020.

Sprint и Scrum догађаји

Пример Sprint-а од две недеље и највећа трајања догађаја према Scrum водичу.

1. недеља					2. недеља				
дан 1	дан 2	дан 3	дан 4	дан 5	дан 6	дан 7	дан 8	дан 9	дан 10
Planning Daily	Daily	Daily	Daily	Daily	Daily	Daily	Daily	Daily	Daily **Review** **Retro**

Sprint	Sprint Planning	Daily Scrum	Sprint Review	Retrospective
≤ 1 месец	≤ 8 h	15 мин	≤ 4 h	≤ 3 h

Наведена трајања важе за Sprint од месец дана. За краћи Sprint догађаји су по правилу краћи.

Daily Scrum

Кратак дневни састанак на коме тим проверава напредак ка циљу Sprint-а.



УЧЕСНИЦИ	Developers, а Scrum Master и Product Owner ако раде на ставкама Sprint-а
ВРЕМЕ	Сваког радног дана, у исто време и на истом месту
СВРХА	Провера напретка ка Sprint Goal-у и план рада за наредна 24 сата

ЧЕСТ ФОРМАТ

1. Шта сам урадио од прошлог састанка?
2. Шта радим до следећег састанка?
3. Шта ме спречава у раду?

⚠ Daily Scrum није извештај руководиоцу. Проблеми се решавају после састанка, са онима којих се тичу.

Преглед и ретроспектива Sprint-а

Sprint се завршава провером производа, а затим провером начина рада.

Sprint Review

шта је направљено

Тим приказује Increment заинтересованим странама на радној верзији система.

Прикупља се повратна информација о урађеном.

Product Backlog се прилагођава новим сазнањима.

Sprint Retrospective

како је тим радио

Тим разматра сарадњу, процес, алате и Definition of Done.

Издвајају се добре праксе и проблеми из протеклог Sprint-а.

Бирају се конкретна побољшања за наредни Sprint.

ЧЕСТ ФОРМАТ РЕТРОСПЕКТИВЕ

Наставити	Престати	Започети

Структура backlog-а и Scrum артефакти

Ставка спремна за рад садржи циљ, приоритет, критеријуме и позната ограничења.

01

Product Backlog

све будуће ставке, уређене по приоритету

Обавеза: Product Goal

02

Sprint Backlog

изабрани обим рада и план за текући Sprint

Обавеза: Sprint Goal

03

Increment

реализована, проверена и употребљива промена

Обавеза: Definition of Done

01 · AGILE И SCRUM

Definition of Ready и Definition of Done

Тим унапред договара када ставка може да уђе у рад и када се сматра завршеном.

Ready

договор тима

- ✓ Опис и вредност за корисника су јасни
- ✓ Критеријуми прихватања су написани
- ✓ Ставка је довољно мала да стане у Sprint
- ✓ Нема зависности које блокирају почетак

Done

обавеза Increment-а

- Критеријуми прихватања су испуњени
- Build и тестови успешно пролазе
- Code review је завршен
- Промена је интегрисана у главну грану
- Документација је ажурирана

02

ДРУГИ ДЕО

Захтеви

Како се потреба корисника записује као корисничка прича и како се дефинишу проверљиви критеријуми прихватања.

Корисничка прича | INVEST | Критеријуми прихватања | Given–When–Then

Структура корисничке приче

Корисничка прича наводи корисника, потребу и очекивану вредност.

US-05 **Резервација опреме**

Као наставник лабораторије,

желим да резервишем слободну опрему за одређени термин,

како бих знао да је опрема доступна и спречио двоструку резервацију.

Корисник
ко има потребу

Потреба
шта корисник жели да уради

Вредност
зашто је то кориснику важно

НАПОМЕНА

Корисничка прича има вредност када дефинише проверљиво понашање система.

02 · ЗАХТЕВИ

Особине добре приче: INVEST

Шест провера које помажу да ставка буде спремна за рад.

I — Независна
`Independent`
реализује се без чекања друге приче

N — Преговарачка
`Negotiable`
детаљи се договарају са тимом

V — Вредна
`Valuable`
доноси јасну корист кориснику

E — Процењива
`Estimable`
тим може да процени обим рада

S — Мала
`Small`
може се завршити у једном Sprint-у

T — Проверљива
`Testable`
има јасне критеријуме прихватања

Извор: B. Wake, „INVEST in Good Stories, and SMART Tasks”, 2003.

02 · ЗАХТЕВИ

Критеријуми прихватања

Сваки критеријум треба да буде проверљив тестом или демонстрационим сценаријем.

ТИП СЦЕНАРИЈА	ПРИМЕР КРИТЕРИЈУМА
Успешан ток	Резервација се креира када је опрема доступна и период исправан.
Пословни конфликт	Систем одбија период који се преклапа са активном резервацијом.
Неисправан улаз	Почетак термина мора бити пре краја.
Гранични случај	Правило јасно дефинише шта важи када се крај једног и почетак другог термина поклапају.

02 · ЗАХТЕВИ

Критеријум као сценарио: Given–When–Then

Почетно стање, акција и очекивани исход директно се преводе у аутоматизовани тест.

ПОСЛОВНИ КОНФЛИКТ

Given	постоји активна резервација опреме од 10:00 до 12:00
When	наставник захтева исту опрему од 11:00 до 13:00
Then	систем одбија захтев и наводи конфликтну резервацију

ГРАНИЧНИ СЛУЧАЈ · ПРИМЕР ПРАВИЛА

Given	постоји активна резервација опреме од 10:00 до 12:00
When	наставник захтева исту опрему од 12:00 до 13:00
Then	резервација се креира, јер се термини само додирују

</> Сваки сценарио постаје један тест: **Given** припрема податке, **When** позива операцију, **Then** проверава исход.

ТРЕЋИ ДЕО

03

Backlog и табла

Како се ставка описује, разлаже на кораке и прати кроз стања рада на тимској табли.

- Садржај ставке
- Разлагање задатка
- Стања рада
- Tapiz Boards

Садржај backlog ставке

Опис, критеријуми, приоритет и подаци о одговорности чувају се уз ставку.

US-05 **Резервација опреме**

01	Опис	Као наставник лабораторије…
02	Критеријуми	4 сценарија
03	Приоритет · Sprint	висок · Sprint 1
04	Assignee	члан тима
	Коментари	договор о граничном случају

01 **Опис**
шта се тражи и ко добија вредност

02 **Критеријуми**
како се доказује исправност

03 **Приоритет и Sprint**
зашто се ради баш сада

04 **Assignee и коментари**
ко ради и шта је договорено

Разлагање задатка

Контролна листа смањује ризик и чини напредак видљивим.

ПРИМЕР КОРАКА 2 / 4

- [x] ~~wireframe или ток екрана~~
- [x] ~~имплементација основне логике~~
- [ ] тестови кључног правила
- [ ] code review и дорада

ЕФЕКТИ

- **мање ризика**
- **видљив застој**
- **лакша провера**
- **лакше образложење**

Стања у развојном току

Статус картице одговара стварном стању посла.

01 **Backlog** — идеја → 02 **Ready** — спремно → 03 **In Progress** — рад → 04 **Code Review** — преглед → 05 **Done** — прихваћено

Backlog и Ready: ставка чека, Ready значи да испуњава Definition of Ready

In Progress и Code Review: рад је у току на feature грани

Done: Definition of Done

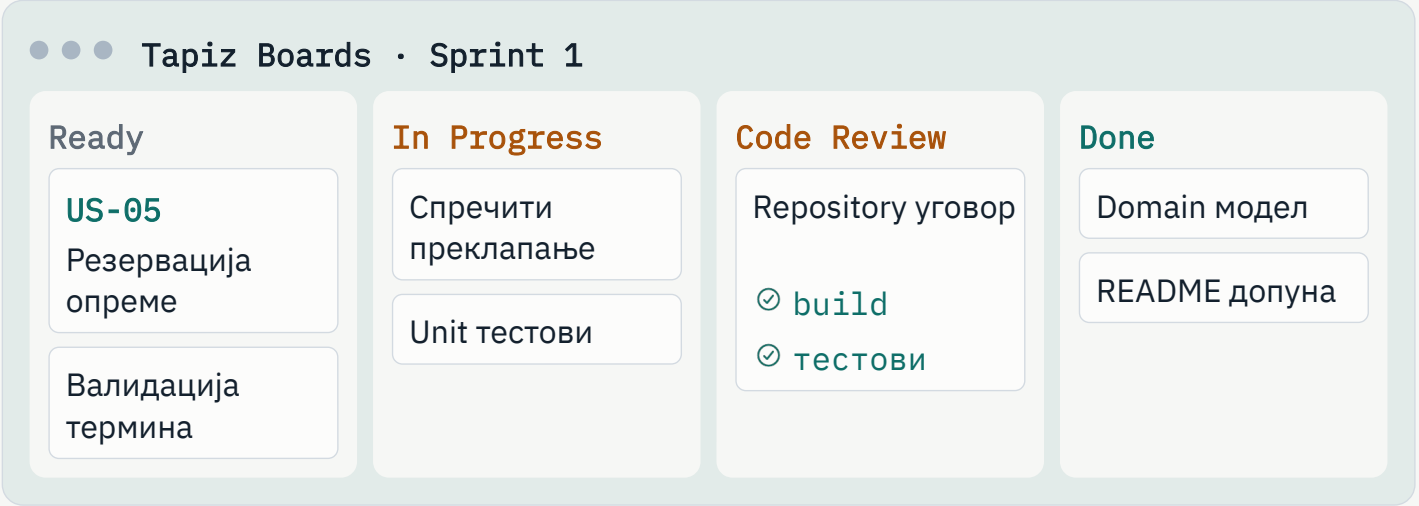
НАПОМЕНА

Статус Done захтева испуњене критеријуме, успешно извршене провере и интегрисану промену.

03 · BACKLOG И ТАБЛА

Tapiz Boards као договор о процесу

Табла приказује ток рада од идеје до прихваћене промене.



КАКО СЕ ЧИТА ТАБЛА

- Колоне показују стварно стање рада, не жељу тима.
- Картица има опис, приоритет, assignee и критеријуме прихватања.
- Веза са PR-ом и тестовима показује да је промена проверљива.
- Done значи прихваћено по Definition of Done, не само померено у последњу колону.

ЧЕТВРТИ ДЕО

04

Git и тимски рад

Историја измена, гране и преглед кода, од локалне измене до интеграције у главну грану.

Појмови | Ток измене | Мали развојни корак | Гране | Pull Request | Конфликти | Интеграција

Git историја измена

Историја развоја омогућава реконструкцију малих и смислених промена.

Појам	Улога у раду тима
Working tree	датотеке које се тренутно мењају
Staging area	избор садржаја за наредни commit
Commit	кохерентна промена са описом намере
Branch	издвојена линија развоја
Remote	удаљени репозиторијум за размену промена

Git као дистрибуирани систем

Git прати измене над скупом датотека и чува историју развоја.

Историја промена се прати локално, а размењује преко удаљеног репозиторијума

Commit има јединствени идентификатор објекта, уобичајено SHA-1 hash ID

Сваки пуни клон има сопствену историју, сервер координише размену измена

Преглед Git команди

Основне команде из полазног материјала, груписане према намени.

Почетак рада	`init` · `clone` · `config`
Локална историја	`add` · `commit` · `commit --amend` · `log`
Гране и интеграција	`checkout` · `rebase` · `merge`
Размена измена	`fetch` · `pull` · `push`

git add

Припрема изабраног садржаја за наредни commit.

Измене прелазе у staging area, односно index

Могу се припремити све измене у текућем директоријуму или појединачна датотека

terminal

```
git add .
git add <ime_datoteke>
git diff --staged
```

git commit

Снимање припремљених измена у локалну историју.

Commit обухвата садржај који је припремљен командом git add

Порука описује једну смислену измену

Креиран commit не мора одмах бити отпремљен на удаљени репозиторијум

terminal

```
git commit -m "izmena"
```

git push

Слање локалних commit-ова на одговарајућу удаљену грану.

Након push-а commit-ови постају доступни осталим сарадницима

Прво објављивање гране поставља везу са origin

terminal

```
git push -u origin feature/moja-izmena
```

Принудно слање преписује историју, потребан је договор тима

terminal

```
git push -u origin feature/moja-izmena --force
```

git fetch

Преузимање објеката и ажурирање референци удаљених грана.

Преузима промене доступне на удаљеном репозиторијуму

Не спаја их аутоматски у текућу грану и не мења радне датотеке

Омогућава преглед доступних измена пре интеграције

terminal

```
git fetch origin
git log --oneline HEAD..origin/main
```

git merge

Интеграција историје изабране гране у текућу грану.

После fetch-а, merge може интегрисати преузете измене

При конфликту треба ускладити садржај, припремити датотеке и завршити спајање

Merge је један од начина интеграције у команди pull

terminal

```
git fetch origin
git merge origin/main
```

git pull

Преузимање, а затим интеграција измена у текућу грану.

Први корак је fetch, други корак зависи од изабраног начина интеграције

У наставном примеру експлицитно бирамо merge поступак

terminal

```
git pull --no-rebase origin main
git pull --rebase origin main
git pull --ff-only
```

git clone

Креирање локалне копије постојећег репозиторијума.

Уобичајени пуни клон преузима датотеке и историју измена

Приватни репозиторијум може се клонирати уз одговарајући приступ

Сарадници и подешавања GitHub платформе управљају се засебно

terminal

```
git clone <url_repozitorijuma>
```

git init, config и log

Креирање репозиторијума, идентитет аутора и преглед историје.

terminal

```
git init
git config --global user.name "Ime Prezime"
git config --global user.email "ime@example.com"
git log --oneline --graph --all
```

Нови локални репозиторијум добија удаљену адресу засебним подешавањем remote-а

checkout и rebase

Промена текуће гране и поновна примена commit-ова на нову основу.

terminal

```
git checkout feature/moja-izmena
git switch feature/moja-izmena
git fetch origin
git rebase origin/main
```

Rebase користити на сопственој грани, уз разумевање последица по историју

git commit --amend

Замена последњег commit-а новим commit-ом.

Користи се за допуну последње локалне измене или исправку поруке

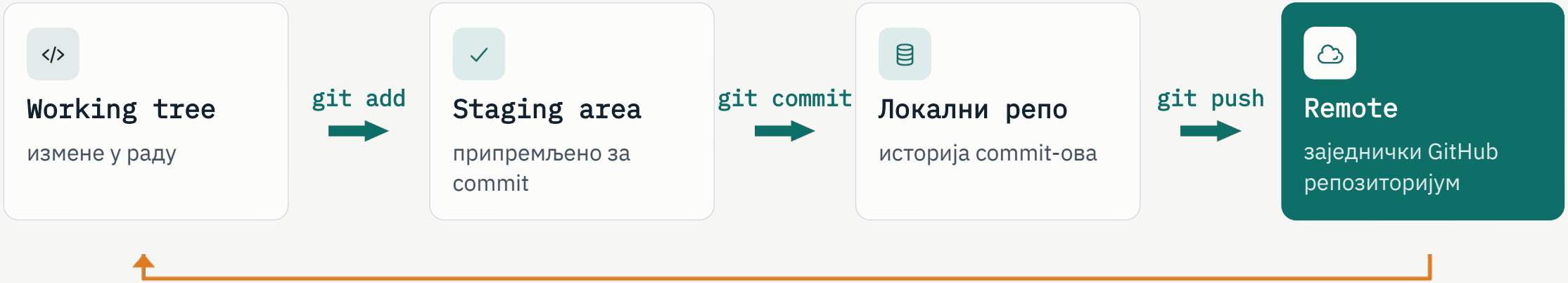
Нови commit замењује претходни и има нов hash ID

terminal

```
git add <ime_datoteke>
git commit --amend --no-edit
```

Пут једне измене кроз Git

Свака команда помера измену за један корак ближе заједничком репозиторијуму.



`git pull` преузима измене командом fetch, а затим их интегрише поступком merge или rebase, према изабраној опцији.

Пример малог развојног корака

Команде су део договореног поступка припреме и провере измене.

terminal · feature/reservation-overlap

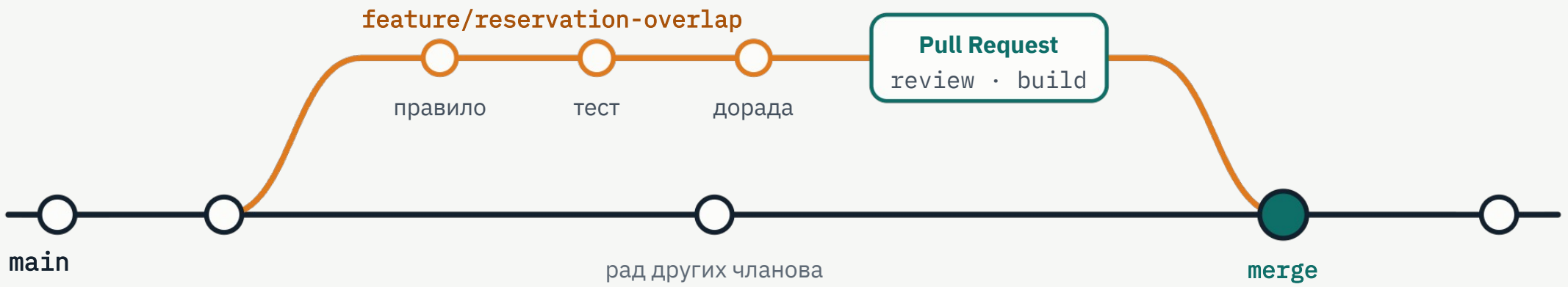
```
git status
git diff
git add src/Application/ReservationService.cs
git add tests/ReservationServiceTests.cs
git diff --staged
git commit -m "Add reservation overlap rule"
git push -u origin feature/reservation-overlap
```

НАПОМЕНЕ

- Пре commit-а прегледати припремљени diff.
- Један commit има једну кохерентну намеру.
- Неповезане промене не иду у исти Pull Request.
- Главна грана остаје стабилна за цео тим.

Рад на feature грани

Свака ставка се развија на сопственој грани, а у главну грану улази тек после прегледа.



- Једна ставка, једна грана
- Мали и смислени commit-и
- main се мења само преко Pull Request-а

04 · GIT

Pull Request и преглед измене

Преглед кода је обавезан део развојног поступка.

PR **Add reservation overlap rule** `feature/reservation-overlap → main` US-05

01 Обим

Да ли промена решава тачно ону ставку на коју се позива?

02 Архитектура

Да ли се поштују договорене границе између слојева и модула?

03 Тестови

Да ли постоји доказ провере за критеријуме прихватања?

04 Ризици

Шта се мења и шта може поћи по злу после спајања?

Решавање конфликта при спајању

Тим усаглашава очекивано понашање пре завршетка спајања.

terminal

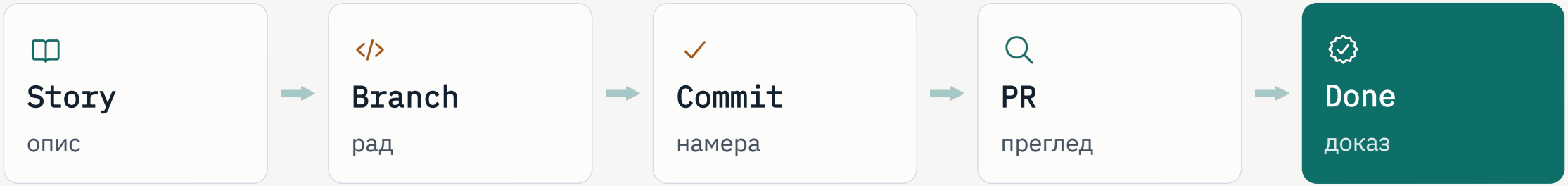
```
git checkout main
git pull
git checkout feature/reservation-overlap
git merge main
# решити конфликт и прегледати diff
git add src/Application/ReservationService.cs
git commit -m "Resolve reservation rule conflict"
dotnet test
```

НАПОМЕНЕ

- Одлучити које понашање мора да остане.
- Последњи корак је провера да систем и даље ради.
- Force push није уобичајен ток на дељеним гранама.

Ток од backlog ставке до интеграције

Сваки корак оставља проверљив траг рада тима.



НАПОМЕНА

Професионални развојни ток повезује алате, договорени процес и проверу резултата.

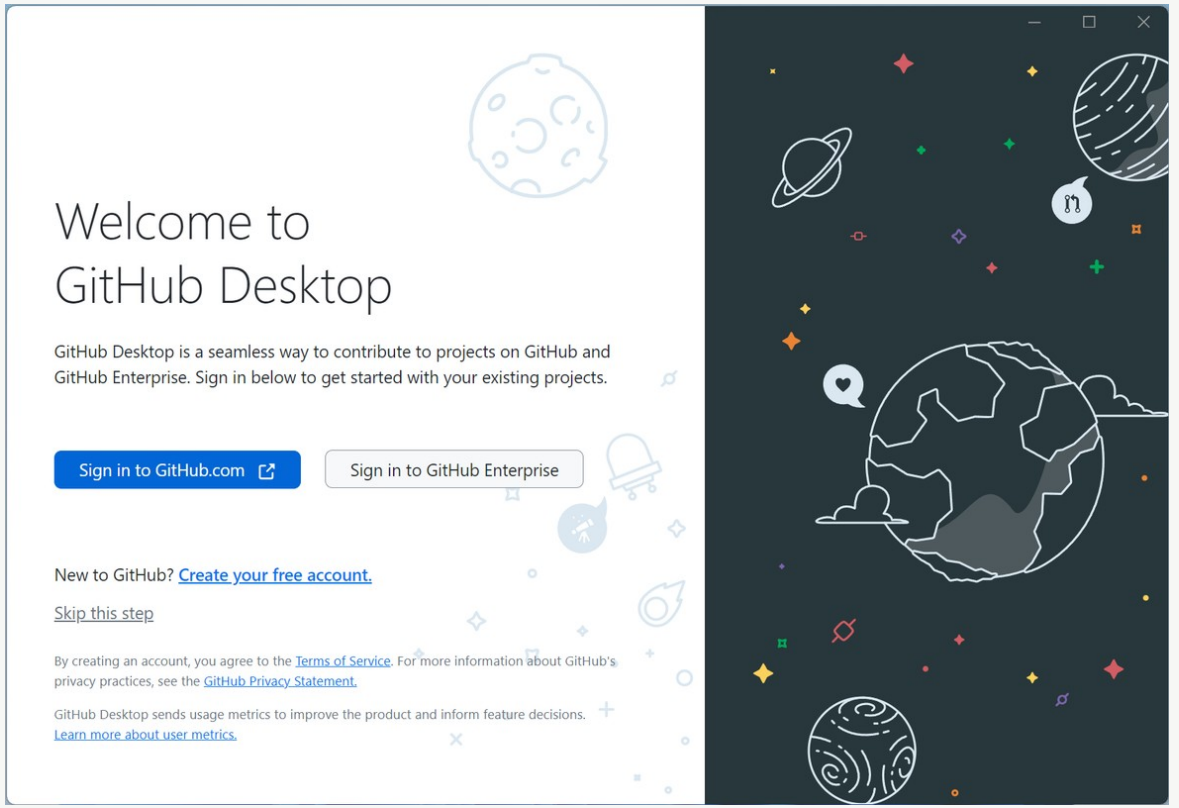
GitHub Desktop

GitHub Desktop је графички Git клијент

Команде су доступне кроз елементе интерфејса

Интеграција са github.com

Преглед измена и аутора



Слика 2.1 · GitHub Desktop

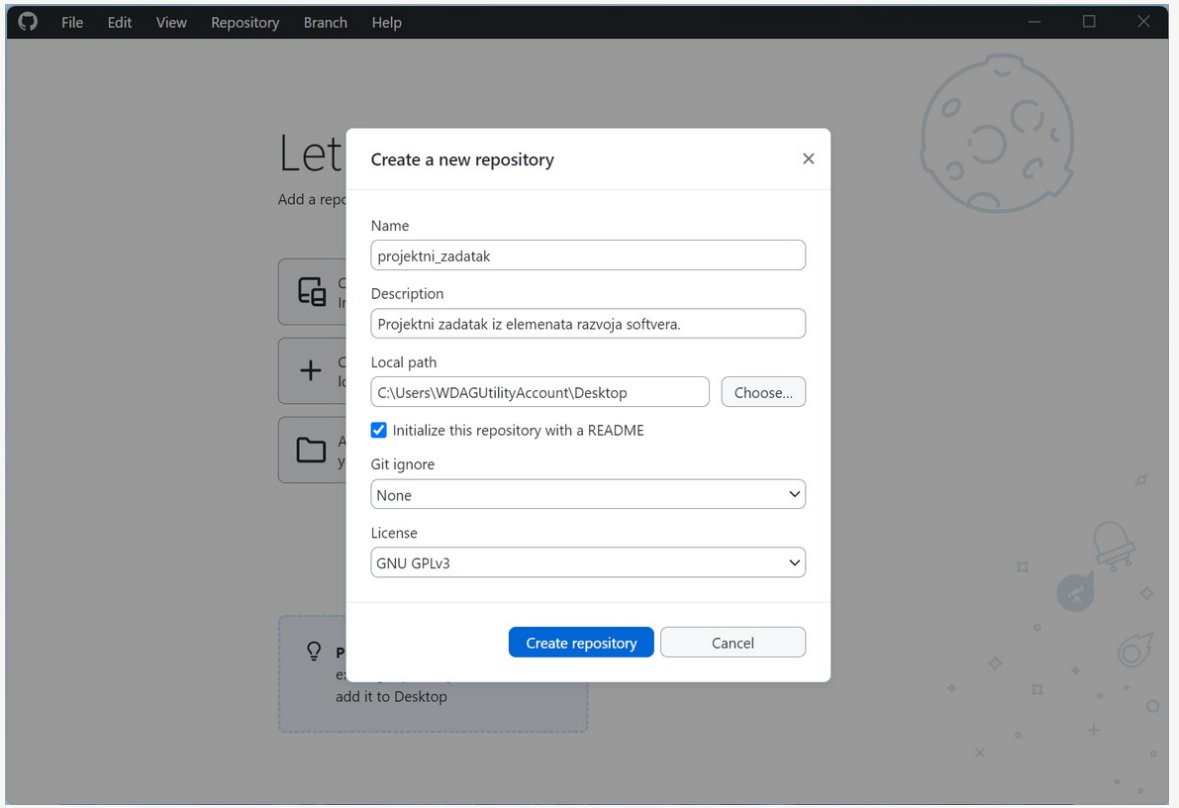
Креирање репозиторијума

Изабрати File → New Repository

Унети основне информације

Одабрати локалну путању

Креирати репозиторијум



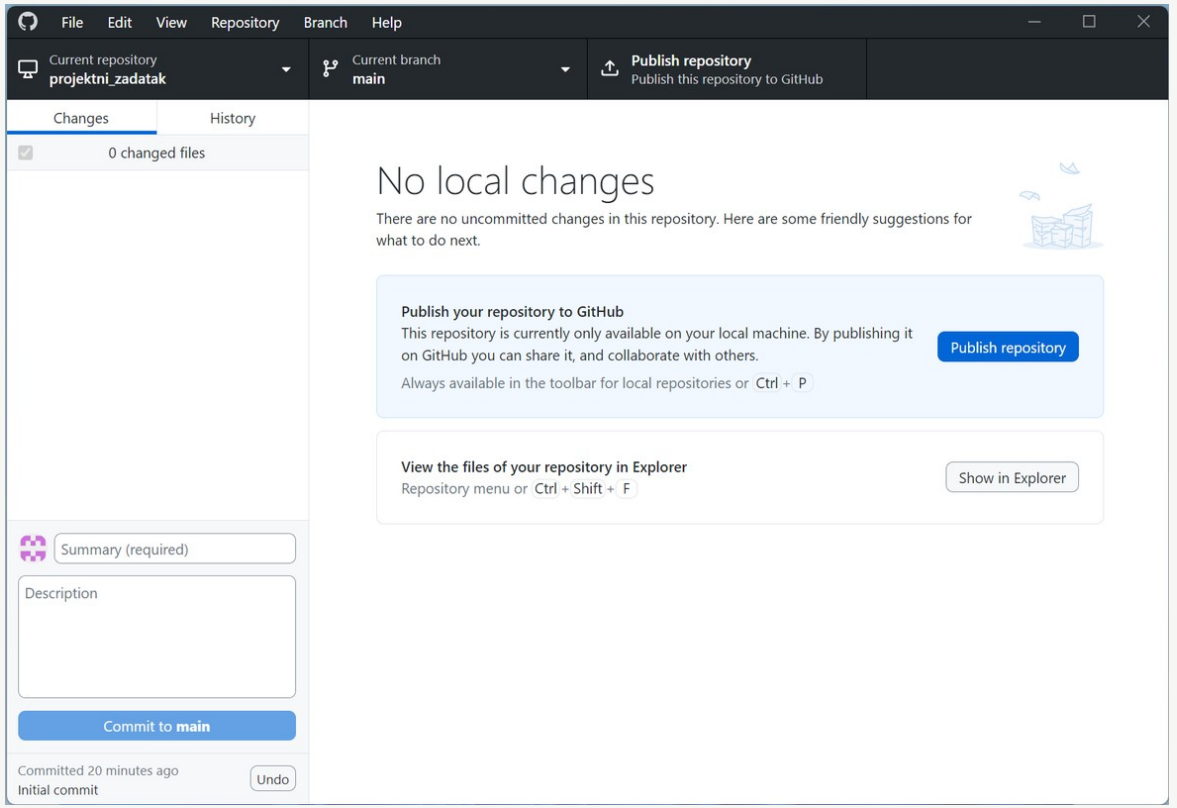
Слика 2.2 · Нови репозиторијум у GitHub Desktop-у

Информације о репозиторијуму

Локални репозиторијум и радне датотеке

Креирање и преглед историје измена

Publish repository објављује репозиторијум на GitHub-у

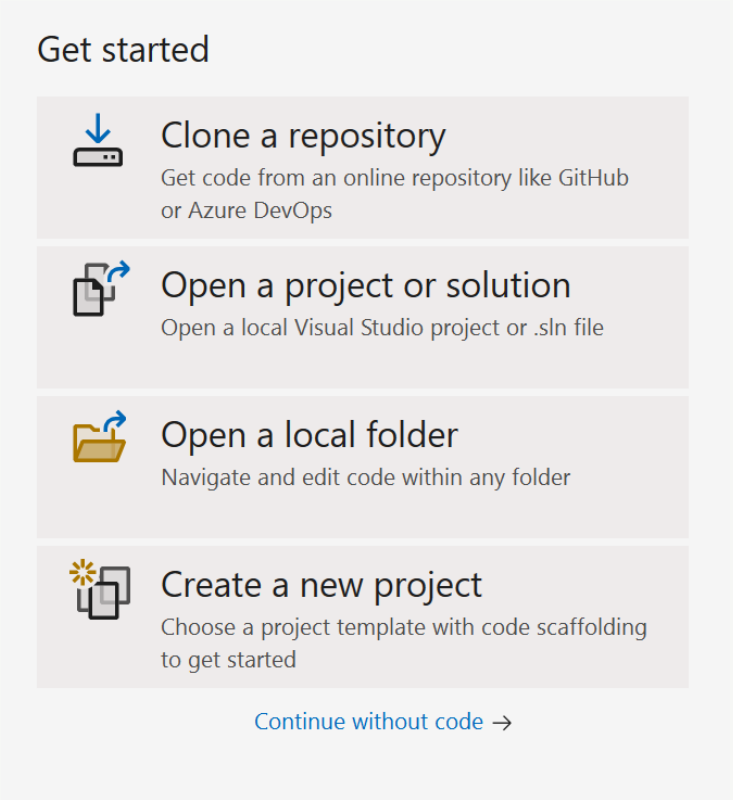


Слика 2.3 · Преглед репозиторијума

Visual Studio и Git

Visual Studio нуди интегрисан рад са Git-ом

За постојећи репозиторијум изабрати Clone a repository



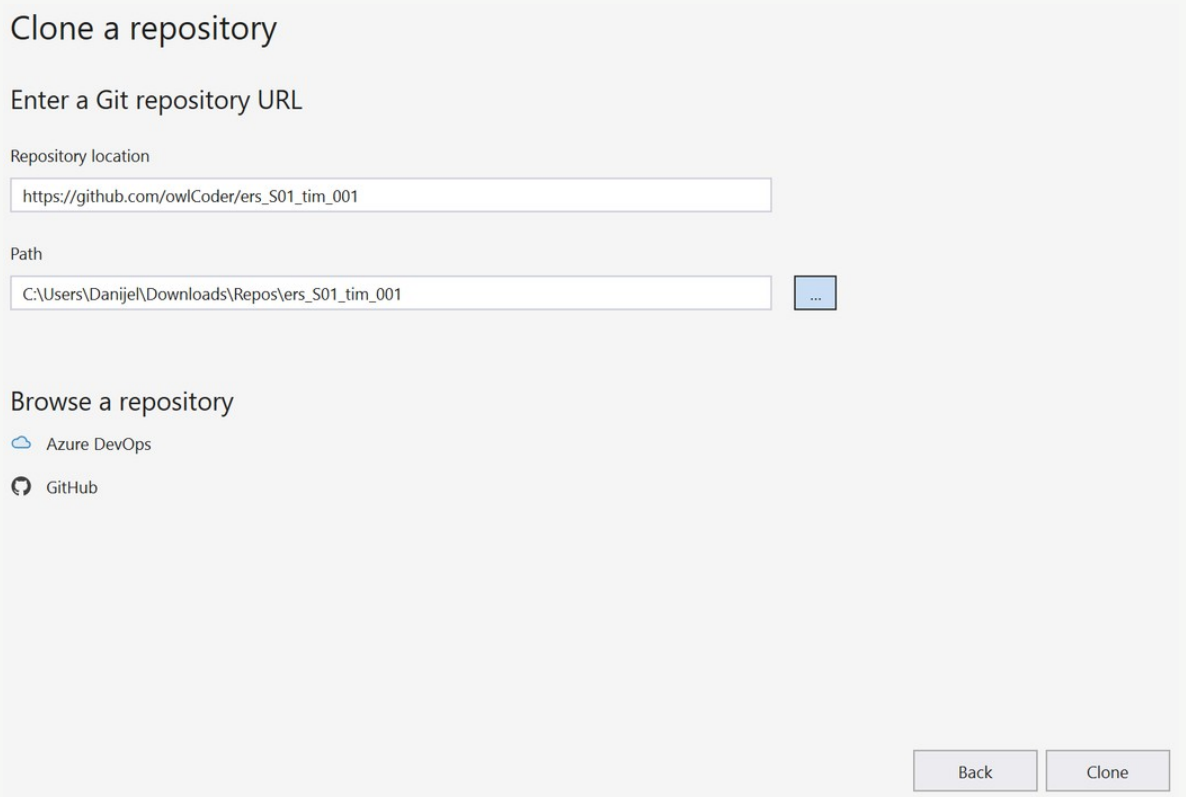
Слика 2.4 · Visual Studio 2022: избор начина отварања

Клонирање у Visual Studio-у

Унети URL репозиторијума и изабрати Clone

Репозиторијум постаје доступан локално

Пријавити се на налог када је приступ репозиторијуму затражен

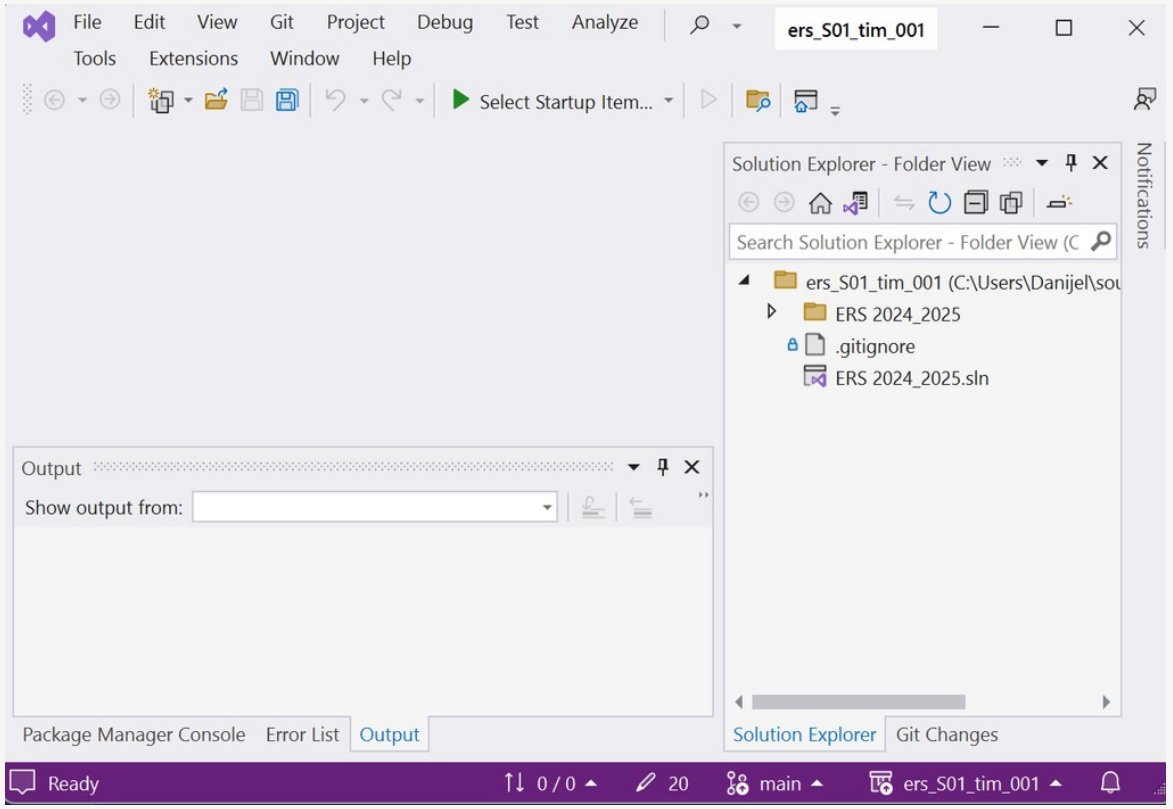


Слика 2.5 · Унос URL-а и локалне путање

Отварање пројекта

Ако је репозиторијум празан, копирати пројекат у његов локални директоријум

Отворити Visual Studio Solution датотеку двоструким кликом



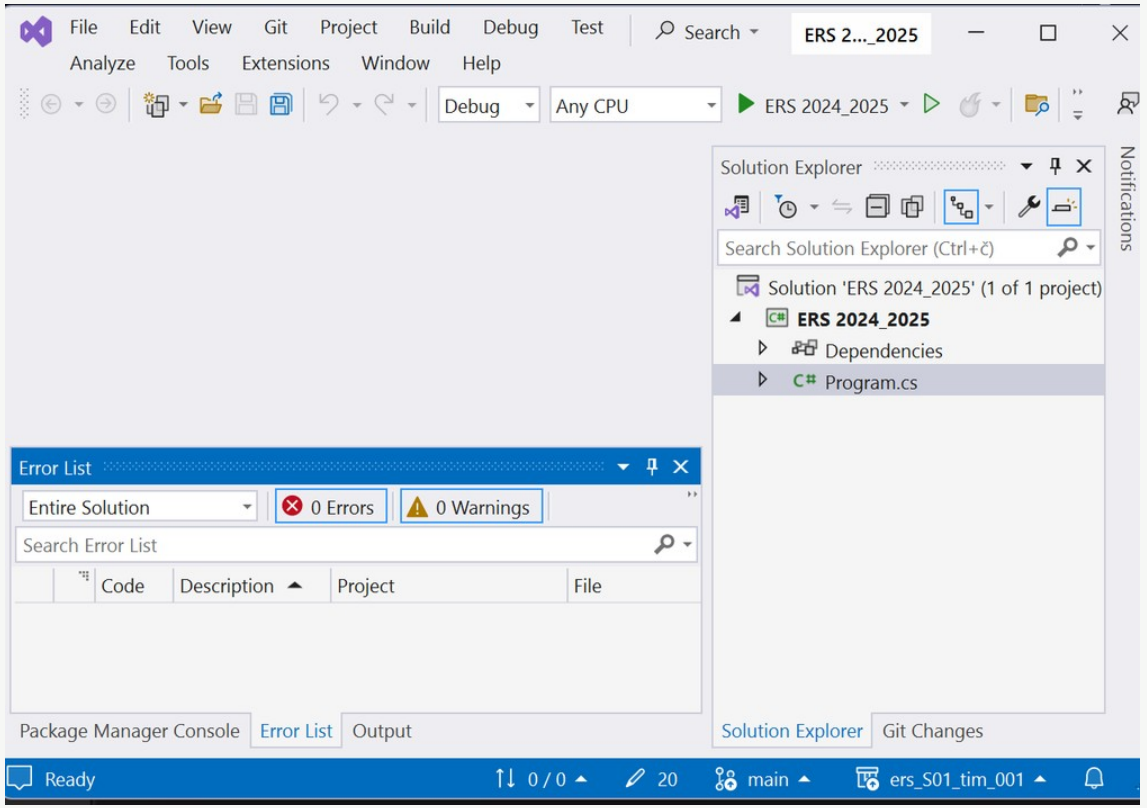
Слика 2.6 · Solution у клонираном репозиторијуму

Преглед измена у Git Changes

За смислен скуп измена формирати commit

Отворити Git Changes

Прегледати измењене датотеке и опције Fetch, Pull и Push



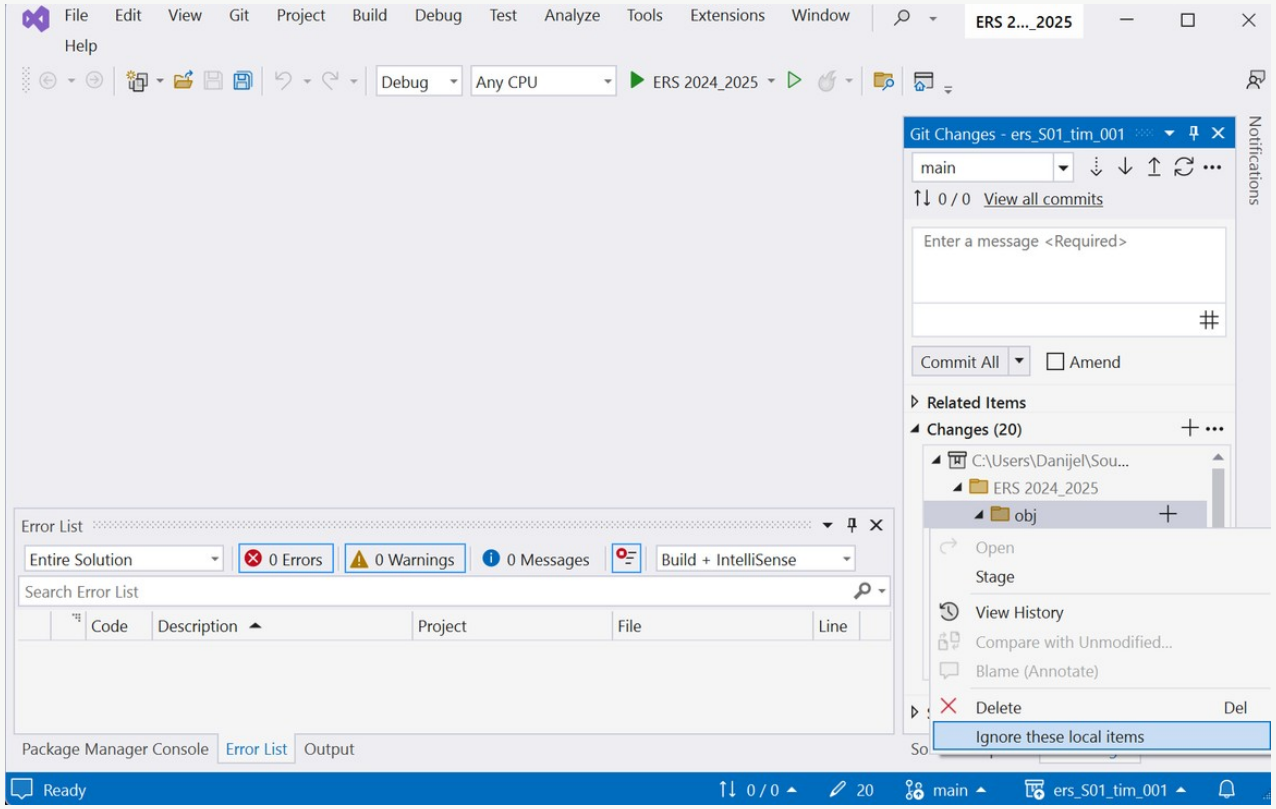
Слика 2.7 · Visual Studio 2022: Git Changes

Игнорисање генерисаних датотека

У .gitignore додати генерисане бинарне и локалне IDE датотеке

Типични директоријуми су bin, obj и .vs

У приказаном интерфејсу: десни клик → Ignore these local items



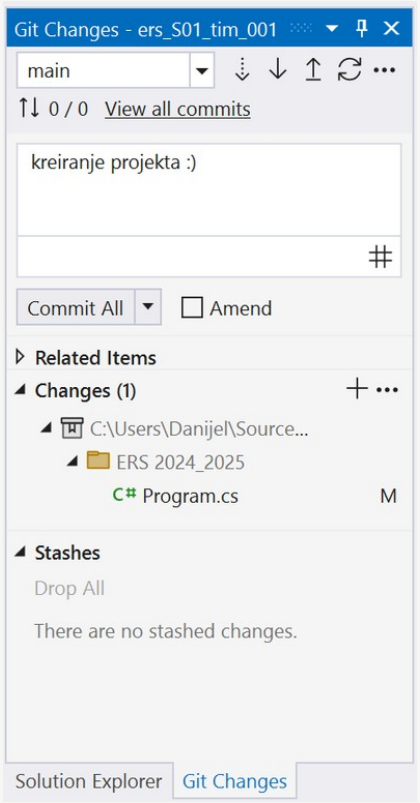
Слика 2.8 · Подешавање .gitignore датотеке

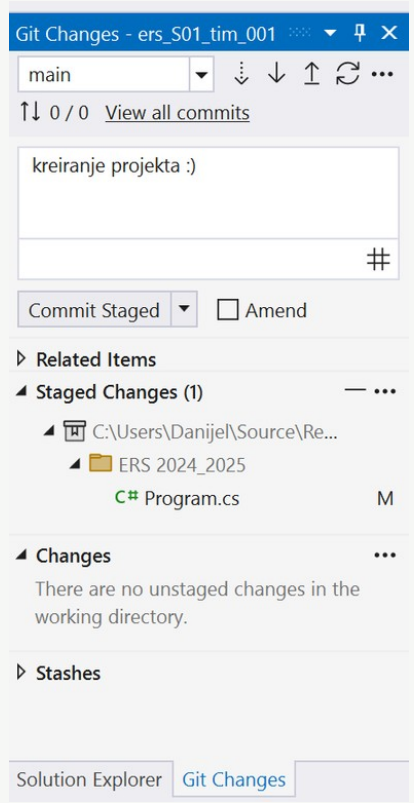
Припрема и креирање commit-а

Кликом на + пренети одабране измене из Changes у Staged Changes

Унети јасну поруку commit-а

Потврдити припремљене измене одговарајућим Commit дугметом





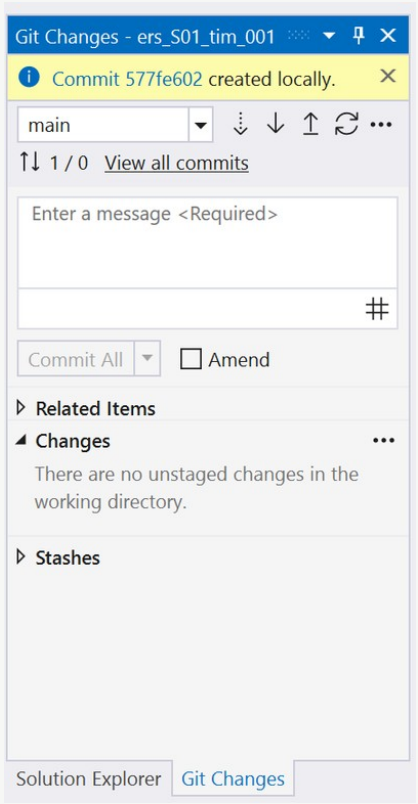
Слика 2.9 · Changes и Staged Changes

Објављивање локалних измена

Commit-ови су сачувани локално

Последњи корак је слање на удаљени репозиторијум

Изабрати Push да измене постану доступне сарадницима



Слика 2.10 · Слање измена из Visual Studio-а

Пројектни репозиторијум

Пример креирања приватног репозиторијума из полазног материјала.

Шаблон назива: ers_XXX_tim_YY

XXX је број пројекта, YY је редни број тима

Као сараднике додати чланове тима и предметног асистента



05

ПЕТИ ДЕО

Предаја и КТ1

Шта се проверава на првој контролној тачки, како се користе AI алати и шта тим проверава пре предаје.

Контролна тачка КТ1 | AI алати | Провера пре предаје | Типични пропусти | Резиме | Припрема

05 · ПРЕДАЈА И КТ1

Контролна тачка КТ1

10 поена

КТ1 у термину Вежбе 4: основна имплементација, извршив ток и тест.

01 **Репозиторијум и README**
README и приступ свих чланова тима

02 **Backlog у Tapiz Boards**
приоритети и критеријуми прихватања

03 **Pull Request**
преглед измена и образложени коментари

04 **Договорени ток**
од Backlog до Done, кроз Review и QA

Одговорност студента при употреби AI алата

AI алати су дозвољени уз проверљив лични допринос и разумевање решења.

ПОТРЕБНО ЈЕ РАЗГРАНИЧИТИ

- **сопствени рад**
- **рад осталих чланова тима**
- **садржај добијен спољним алатима**

РЕЛЕВАНТАН ДОКАЗ РАДА

- репозиторијум и историја измена
- тестови и документација
- евиденција употребе AI алата
- самостално образложење

Провера пре предаје

Тим проверава основне услове пре означавања рада као завршеног.

- [ ] 01 Да ли је ставка повезана са граном или Pull Request-ом?
- [ ] 02 Да ли критеријуми прихватања имају видљиву проверу?
- [ ] 03 Да ли diff садржи само релевантне измене?
- [ ] 04 Да ли је главна грана стабилна?
- [ ] 05 Да ли сваки члан може да објасни свој допринос?

05 · ПРЕДАЈА И КТ1

Типични пропусти

Наведени пропусти нарушавају следљивост и отежавају проверу рада.

⚠ **Нејасна прича**
није јасно ко добија вредност

⚠ **Без критеријума**
не зна се када је ставка завршена

⚠ **Велики Pull Request**
преглед губи фокус

⚠ **Без доказа**
нема теста, diff-а или објашњења

05 · РЕЗИМЕ

Циљ прве вежбе је успостављање дисциплинованог и следљивог развојног поступка.

Професионалан тимски рад почиње када се захтев може пратити до кода, теста, review-a и интеграције.

захтев → код → тест → review → интеграција

Ови резултати представљају основу за наредне вежбе.

05 · ПРЕДАЈА И КТ1

Припрема за наредну вежбу

Резултати прве вежбе користе се у даљем раду на пројекту.

01

Написати User Story за једну ставку пројектног домена.

02

Додати успешне, негативне и граничне критеријуме прихватања.

03

Увести ставку у Tapiz ток и радити преко feature гране.

04

Отворити Pull Request са доказом провере.

05

Објаснити цео пут од захтева до интегрисане промене.

Ово је основа за контролну тачку КТ1 у термину Вежбе 4.